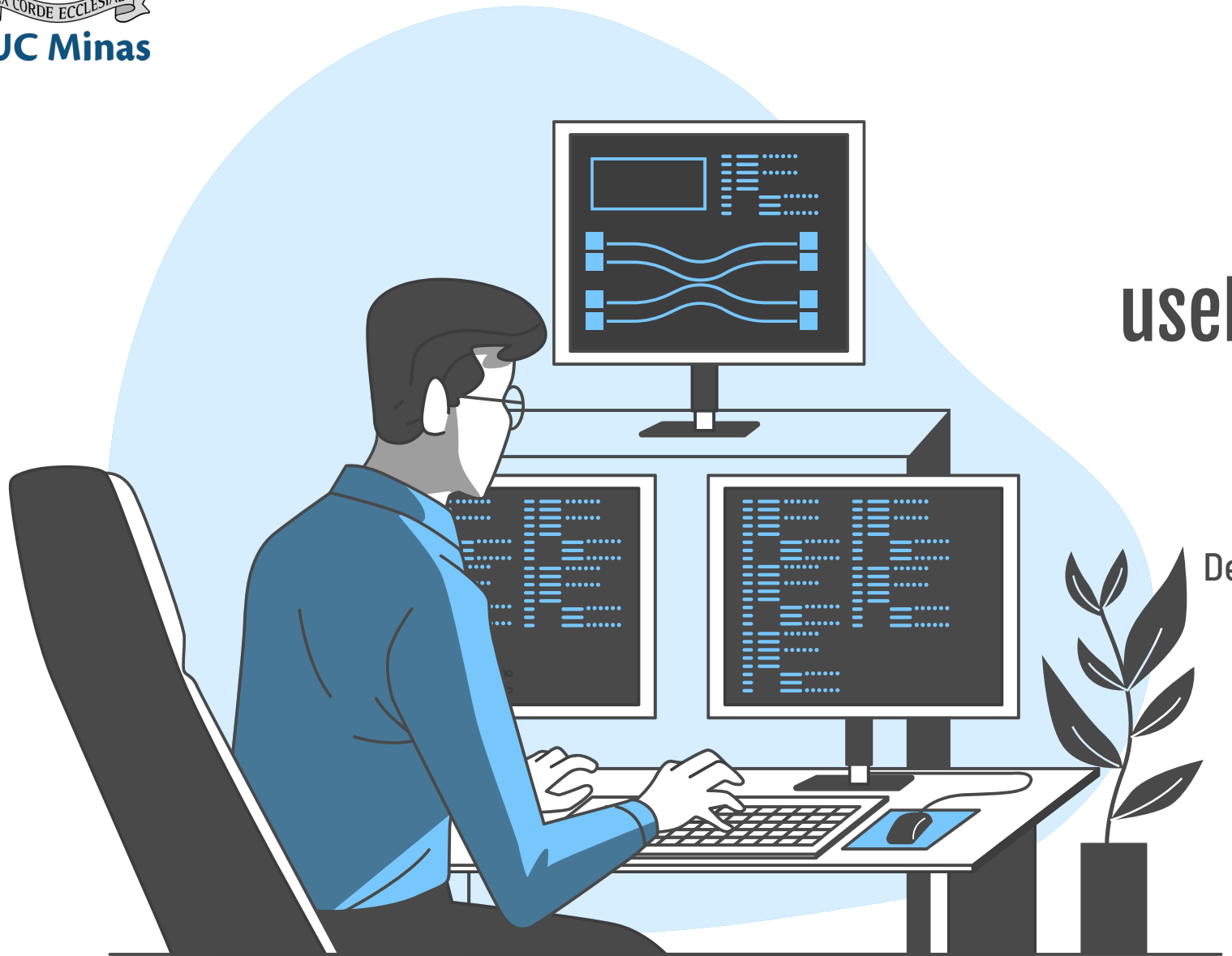
PUC Minas



Aula 06 · Front-end

useEffect e consumo de APIs

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender **efeitos colaterais** e o `useEffect`

2

Controlar quando o efeito roda com o **array de dependências**

3

Consumir APIs com `fetch` e `async/await`

4

Tratar estados de **carregando**, **erro** e **vazio**

5

Separar as chamadas HTTP em `services/`

Antes de começar

- Aula 05: estado

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Efeitos colaterais

02 Buscando dados com fetch

03 A camada de services

04 Enviando dados (POST, PUT, DELETE)

05 Inspecionando no DevTools

06 Aprofundando: requisições robustas

01

Efeitos colaterais

Efeitos colaterais

Renderizar deve ser **puro**: receber props/estado e devolver JSX. Tudo que fala com o mundo de fora (buscar dados, usar `setTimeout`, mexer no `document.title`, assinar eventos) é um **efeito** e vai dentro de `useEffect`, que roda **depois** que a tela é desenhada.

01

Efeitos colaterais

Aula 06 · Front-end

Efeitos colaterais

Forma geral

```
useEffect(() => {
  // código do efeito
  return () => {
    // limpeza (opcional): roda antes do próximo efeito e quando o componente sai da tela
  };
}, [dependencias]);
```

Efeitos colaterais

Array de dependências	Quando o efeito roda
`[]`	Uma vez, quando o componente aparece (buscar dados iniciais)
`[busca]`	Na primeira vez e sempre que `busca` mudar
(omitido)	Depois de **toda** renderização (raramente o que você quer)

Efeitos colaterais

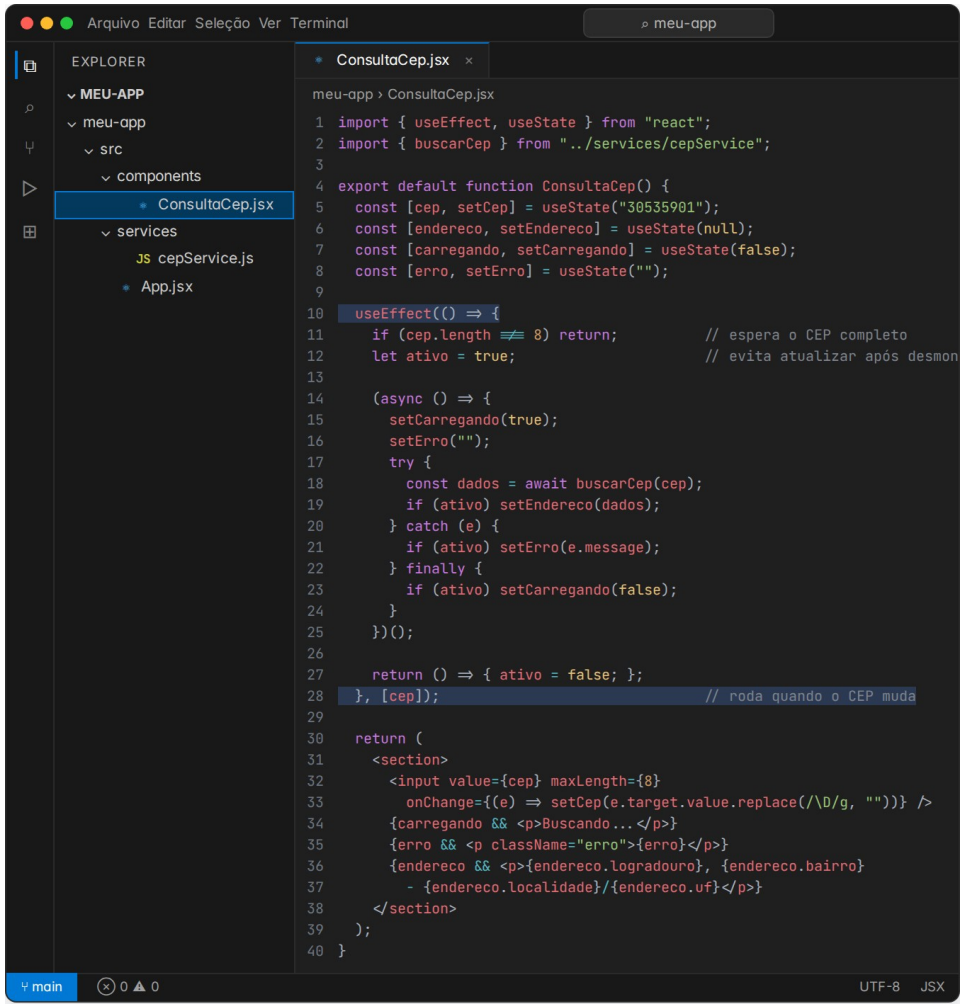
Observação

Em desenvolvimento, o `StrictMode` monta, desmonta e monta de novo cada componente, então um efeito com `[]` roda **duas vezes**. É proposital, para revelar efeitos sem limpeza. Em produção roda uma vez.

02

Buscando dados com fetch

Buscando dados com fetch



O `fetch` é nativo do navegador e devolve uma *Promise*. Com `async/await` o código fica linear. Exemplo usando a API pública do **ViaCEP** (projeto `ConsultaBrasil`):

Componente com os três estados clássicos: dados, carregando e erro.

Buscando dados com fetch

src/services/cepService.js

```
const BASE = "https://viacep.com.br/ws";

export async function buscarCep(cep) {
  const resposta = await fetch(`${BASE}/${cep}/json/`);
  if (!resposta.ok) throw new Error(`Erro ${resposta.status} ao consultar o CEP.`);

  const dados = await resposta.json();
  if (dados.erro) throw new Error("CEP não encontrado.");
  return dados;
}
```

Buscando dados com fetch

Atenção

O `fetch` **só lança erro em falha de rede** (servidor desligado, CORS bloqueado). Respostas `404` ou `500` chegam normalmente: confira sempre `resposta.ok` (verdadeiro para status 200 a 299).

03

A camada de services

A camada de services

Os projetos da disciplina nunca chamam `fetch` direto nos componentes. As chamadas ficam em `src/services/`, e a URL base em `src/config/`:

Pasta	Conteúdo	Exemplo
`config/`	Endereços e chaves lidos do ambiente	`apiConfig.js` com `BASE_URL`
`services/`	Uma função por endpoint	`listarAlunos()`, `criarAluno(dados)`
`components/`	Só tela; chama os services	`AlunoList.jsx`

A camada de services

Vantagem: se a URL ou o formato da API mudar, você altera um arquivo só. Veremos a versão completa, com `httpClient.js`, na pasta FullStack.

03

A camada de services

Aula 06 · Front-end

04

Enviando dados (POST, PUT, DELETE)

Enviando dados (POST, PUT, DELETE)

services/alunoService.js

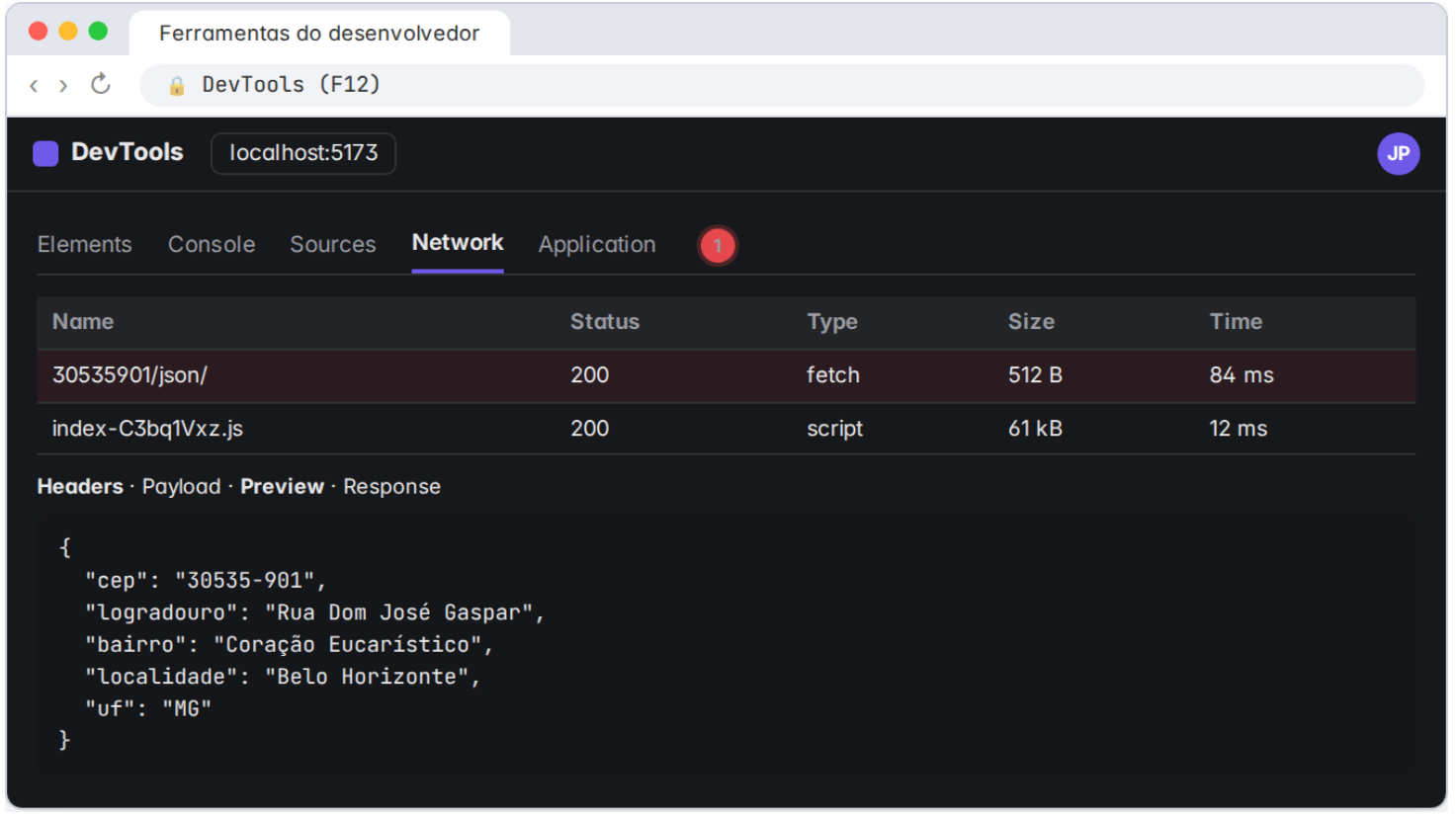
```
export async function criarAluno(aluno) {
  const r = await fetch(`${BASE_URL}/api/alunos`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(aluno),            // objeto → texto JSON
  });
  if (!r.ok) throw new Error((await r.json()).mensagem);
  return r.json();
}

export async function excluirAluno(id) {
  const r = await fetch(`${BASE_URL}/api/alunos/${id}`, { method: "DELETE" });
  if (!r.ok) throw new Error("Não foi possível excluir.");
}
```

05

Inspecionando no DevTools

Inspecionando no DevTools



Aba **Network** do DevTools: toda chamada `fetch` aparece aqui com status, tempo e resposta.

Inspecionando no DevTools

Dica

Filtre a aba Network por **Fetch/XHR** para ver só as chamadas de API. Clique em uma linha e veja **Headers** (URL, método, status), **Payload** (o que foi enviado) e **Response** (o que voltou).

06

Aprofundando: requisições robustas

Cancelando requisições com AbortController

Evita que uma resposta antiga sobrescreva a nova

```
useEffect(() => {
  const controle = new AbortController();

  fetch(`https://api.github.com/users/${usuario}`, { signal: controle.signal })
    .then((r) => r.json())
    .then(setPerfil)
    .catch((e) => { if (e.name !== "AbortError") setErro(e.message); });

  return () => controle.abort();      // usuário digitou outro nome: cancela
}, [usuario]);
```

Debounce em campos de busca

Mesmo padrão do App.jsx do Exemplo_Integracao

```
const [busca, setBusca] = useState("");

useEffect(() => {
  const timer = setTimeout(() => carregarAlunos(busca), 300);  // espera parar de digitar
  return () => clearTimeout(timer);                            // nova tecla: cancela o anterior
}, [busca]);
```

fetch x axios

fetch (nativo)

- Já vem no navegador, zero dependências
- Precisa checar `resposta.ok` e chamar `.json()`
- Usado nos projetos da disciplina

vs

axios (biblioteca)

- `npm install axios`
- Lança erro em status 4xx/5xx e converte JSON sozinho
- Interceptadores para colocar o token em toda requisição

Aprofundando: requisições robustas

Promise.all

```
// várias requisições em paralelo
const [perfil, repos] = await Promise.all([
  fetch(`${API}/users/${u}`).then((r) => r.json()),
  fetch(`${API}/users/${u}/repos`).then((r) => r.json()),
]);
```

Aprofundando: requisições robustas

API pública	Usada no projeto	Exige chave?
ViaCEP / BrasilAPI	`ConsultaBrasil`	Não
PokeAPI	`Pokedex`	Não
GitHub REST API	`GitHubAPI`, `ContributorStats`	Opcional (limite maior com token)
Radio Browser	`RadioBrowserAPI_2_React`	Não
Spotify, Gemini, Stripe	`SpotifyAPI`, `GeminiAPI`, `StripeAPI`	Sim (cuidado: segredo vai no back-end)

Os quatro estados de uma tela que busca dados

Carregando

Mostre um indicador. Evite a tela em branco.

Erro

Mensagem clara e, se possível, um botão “Tentar de novo”.

Vazio

“Nenhum aluno cadastrado” convida a cadastrar o primeiro.

Sucesso

Os dados. Atualize a lista depois de criar/excluir.

Aprofundando: requisições robustas

Observação

Para projetos maiores, bibliotecas como **TanStack Query** cuidam de cache, recarregamento e estados de carregamento automaticamente. Vale conhecer depois de dominar o `useEffect` + `fetch` desta aula.

Aprofundando: requisições robustas

Um hook useBusca reaproveitável (implemente no exercício)

```
export default function Lista() {
  const { dados, carregando, erro, recarregar } = useBusca(listarAlunos);
  if (carregando) return <p>Carregando...</p>;
  if (erro) return <p>{erro} <button onClick={recarregar}>Tentar de novo</button></p>;
  if (dados.length === 0) return <p>Nenhum aluno.</p>;
  return <ul>{dados.map((a) => <li key={a.id}>{a.nome}</li>)}</ul>;
}
```

07

Quiz de revisão

Pergunta

Quando um efeito com array de dependências `[]` executa?

A. A cada renderização

B. Uma vez, ao montar o componente

C. Nunca

D. Ao clicar

Quiz · questão 1 de 5

Resposta

Quando um efeito com array de dependências `[]` executa?

A. A cada renderização

✓ Uma vez, ao montar o componente

C. Nunca

D. Ao clicar

Por quê: Com [x] roda também quando x muda.

Quiz · questão 2 de 5

Pergunta

O `fetch` lança erro quando a API devolve 404?

A Sim

B Não, é preciso checar resposta.ok

C Só no Chrome

D Só com JSON

Resposta

O `fetch` lança erro quando a API devolve 404?

A. Sim

✓ Não, é preciso checar resposta.ok

C. Só no Chrome

D. Só com JSON

Por quê: Ele só falha em erros de rede ou CORS.

Pergunta

Para que serve a função de limpeza retornada pelo useEffect?

A Apagar o banco

B Cancelar timers/requisições antes do próximo efeito ou ao sair da tela

C Limpar o CSS

D Recarregar a página

Resposta

Para que serve a função de limpeza retornada pelo useEffect?

A Apagar o banco

✓ Cancelar timers/requisições antes do próximo efeito ou ao sair da tela

C Limpar o CSS

D Recarregar a página

Por quê: Evita atualizar estado de componente que já saiu.

Pergunta

Por que separar as chamadas HTTP em `services/`?

A Exigência do React

B Mudança na API é feita em um só lugar e os componentes ficam limpos

C Fica mais rápido

D Para evitar CORS

Resposta

Por que separar as chamadas HTTP em `services/`?

A Exigência do React

✓ Mudança na API é feita em um só lugar e os componentes ficam limpos

C Fica mais rápido

D Para evitar CORS

Por quê: Componentes só chamam funções como listarAlunos().

Pergunta

Onde ver a URL, o status e a resposta de cada chamada fetch?

A Aba Elements

B Aba Network do DevTools

C No terminal do Vite

D No package.json

Resposta

Onde ver a URL, o status e a resposta de cada chamada fetch?

A Aba Elements

✓ Aba Network do DevTools

C No terminal do Vite

D No package.json

Por quê: Filtre por Fetch/XHR.

Fechamento

Resumo da aula

- ✓ Busca de dados é **efeito**: vai em `useEffect`, que roda após a renderização.
- ✓ Dependências: `[]` roda uma vez; `[x]` roda quando `x` muda; retorne uma função de limpeza.
- ✓ `fetch` + `async/await`; confira `resposta.ok`, porque 404/500 não lançam erro.
- ✓ Sempre trate **carregando**, **erro** e **vazio**.
- ✓ Chamadas HTTP ficam em `services/`; a URL base em `config/`.

Exercícios

1. Liste os 151 primeiros Pokémon da PokeAPI com nome e imagem.

2. Adicione um campo de busca que filtra a lista, esperando 300 ms após a última tecla (*debounce* com `setTimeout`).

3. Crie `services/githubService.js` e mostre os repositórios de um usuário do GitHub.

4. Simule a API fora do ar (URL errada) e garanta que a tela mostre uma mensagem amigável.

Para ir além

- react.dev/reference/react/useEffect
- developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API
- Projetos `ConsultaBrasil`, `GitHubAPI`, `Pokedex`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 06?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web